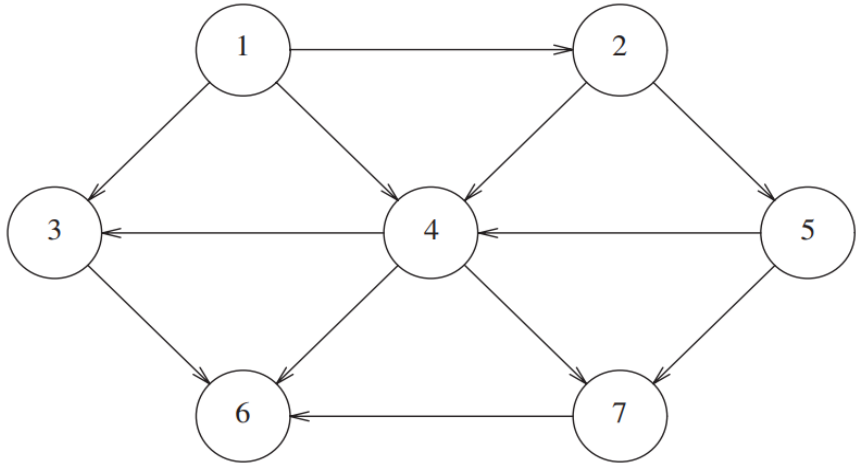
CSE 4303
Data Structures

Topic: Graph Basics

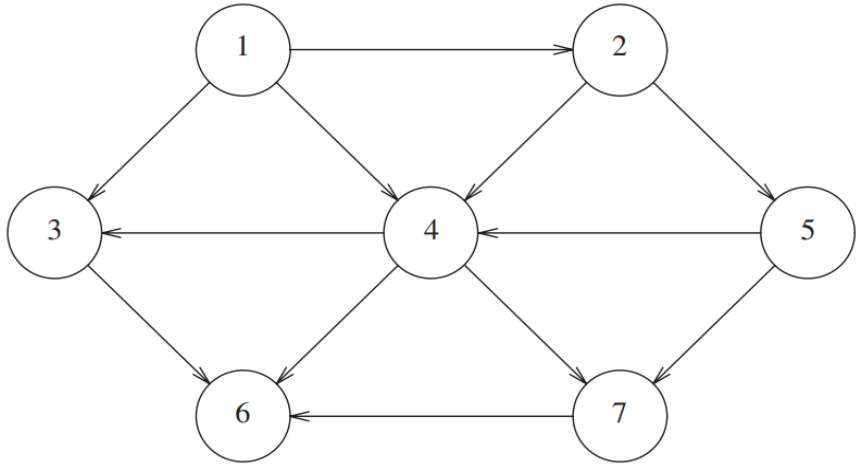
Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Introduction

- A **Graph** $G = (V, E)$ consists of
  - A set of **Vertices** (V)
  - A set of **Edges** (E)

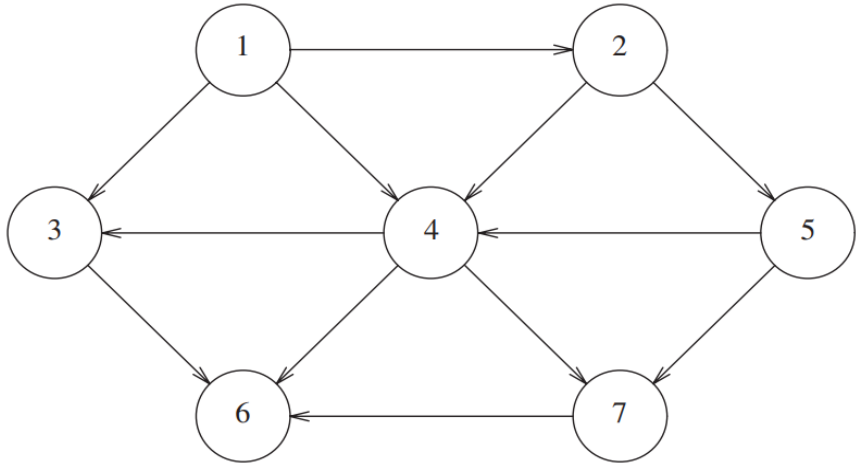


Introduction



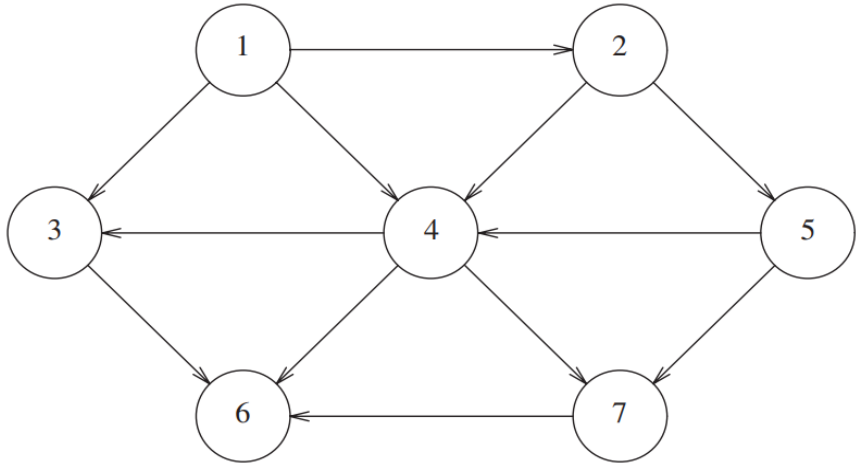
- A **Graph** $G = (V, E)$ consists of
  - A set of **Vertices** (V)
  - A set of **Edges** (E)
- $V = \{v_1, v_2, \dots, v_n\}$, where total number of vertices is $|V| = n$.

Introduction



- A **Graph** $G = (V, E)$ consists of
  - A set of **Vertices** (V)
  - A set of **Edges** (E)
- $V = \{v_1, v_2, \dots, v_n\}$, where total number of vertices is $|V| = n$.
- Each edge is a pair (v, w), where $v, w \in V$.

Introduction

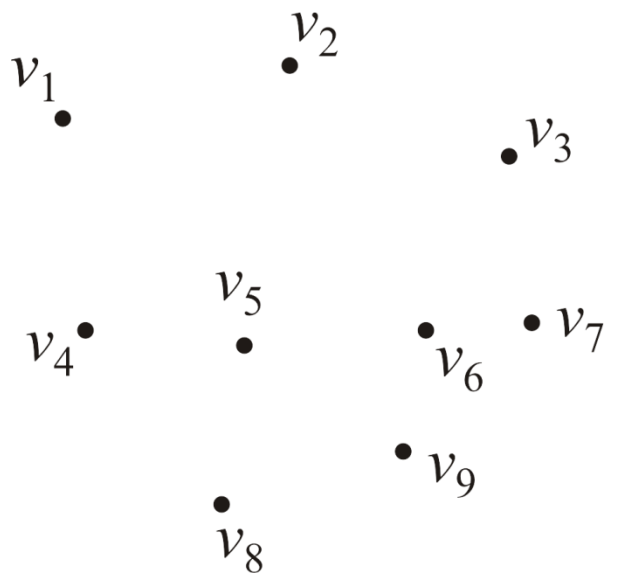


- A **Graph** $G = (V, E)$ consists of
  - A set of **Vertices** (V)
  - A set of **Edges** (E)
- $V = \{v_1, v_2, \dots, v_n\}$, where total number of vertices is $|V| = n$.
- Each edge is a pair (v, w), where $v, w \in V$.

- Graphs can be **directed** or **undirected**.
  - Directed: Edge (v, w) means it starts from v and ends at w.
  - For undirected graph, order is not important.

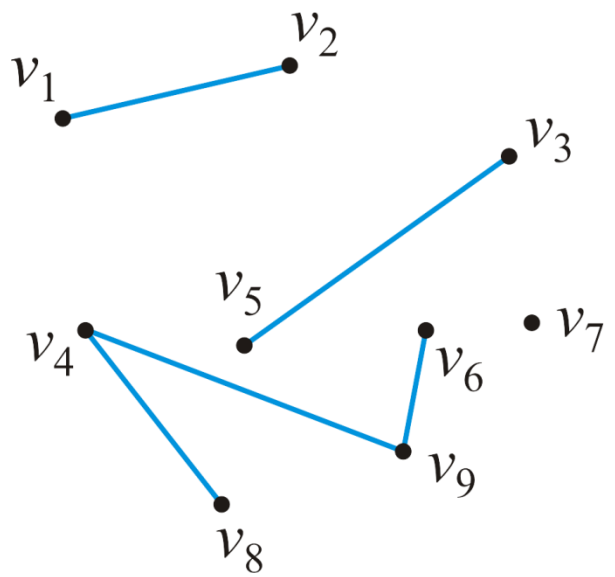
Graphs

- Consider this collection of vertices $\{v_1, v_2, \ldots, v_9\}$



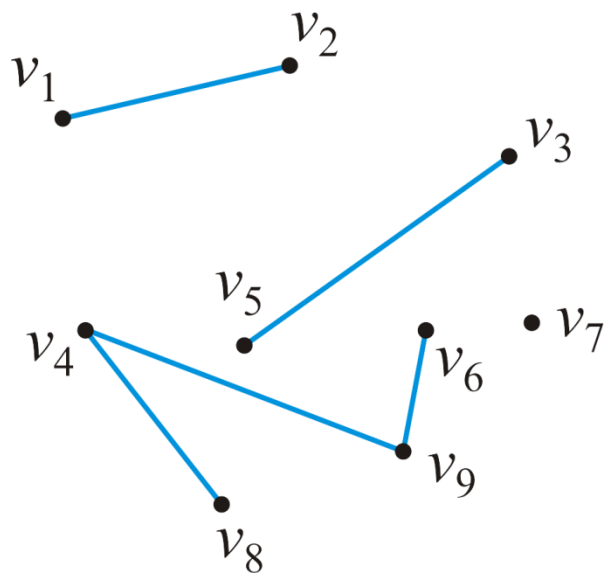
Undirected graphs

- Associated with these vertices are $|E| = 5$ edges
  - $E = \{\{v_1, v_2\}, \{v_3, v_5\}, \{v_4, v_8\}, \{v_4, v_9\}, \{v_6, v_9\}\}$

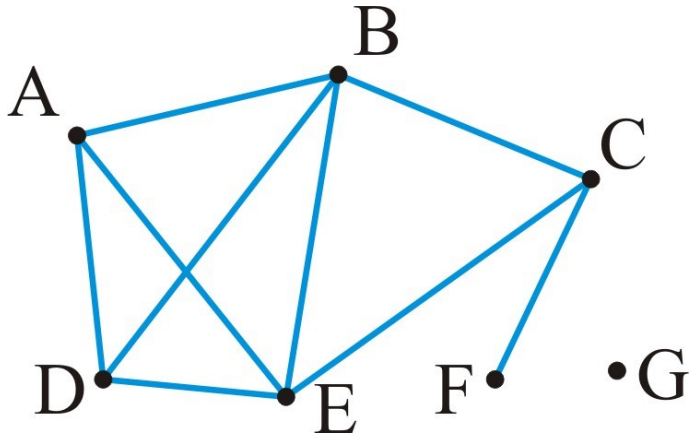


Undirected graphs

- Associated with these vertices are $|E| = 5$ edges
  - $E = \{\{v_1, v_2\}, \{v_3, v_5\}, \{v_4, v_8\}, \{v_4, v_9\}, \{v_6, v_9\}\}$
- Pair $\{v_j, v_k\}$ indicates: both vertex v_j is adjacent to v_k, and vertex v_k is adjacent to v_j.

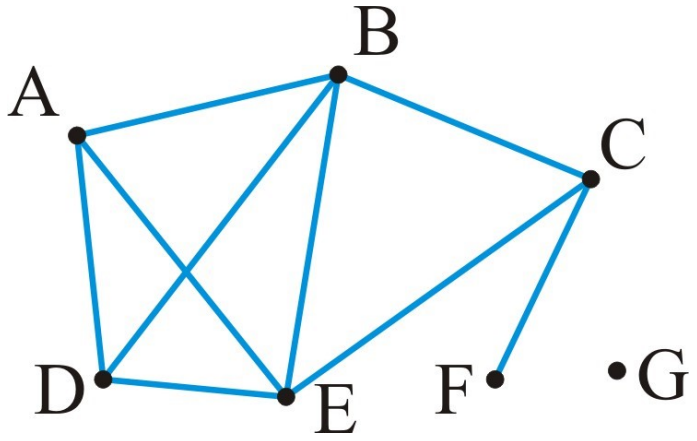


An undirected graph



- Given a graph with
  - $|V| = \{A, B, C, D, E, F, G\}$, 7 vertices
  - $|E| =$ 9 edges
  - $E = \{\{A, B\}, \{A, D\}, \{A, E\}, \{B, C\}, \{B, D\}, \{B, E\}, \{C, E\}, \{C, F\}, \{D, E\}\}$

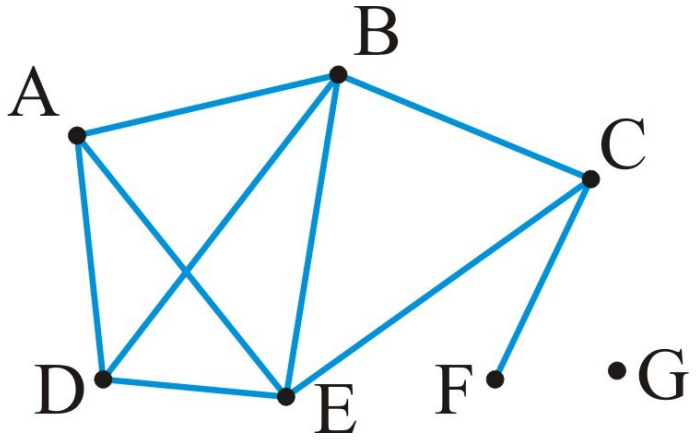
An undirected graph



- Given a graph with
  - $|V| = \{A, B, C, D, E, F, G\}$, 7 vertices
  - $|E| =$ 9 edges
  - $E = \{\{A, B\}, \{A, D\}, \{A, E\}, \{B, C\}, \{B, D\}, \{B, E\}, \{C, E\}, \{C, F\}, \{D, E\}\}$

- The **maximum number of edges** in an undirected graph is
  - ???

An undirected graph

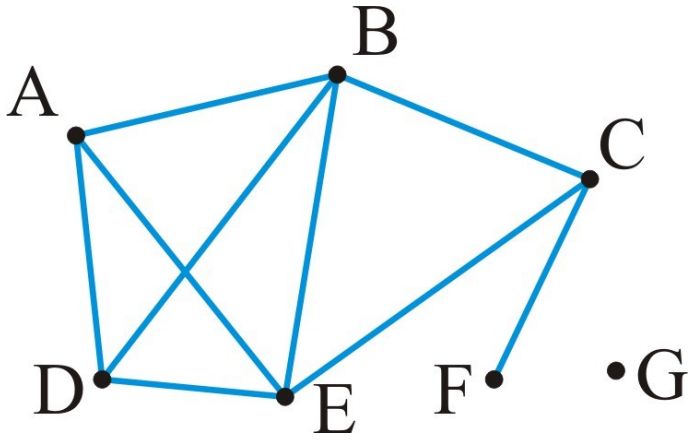


- Given a graph with
  - $|V| = \{A, B, C, D, E, F, G\}$, 7 vertices
  - $|E| =$ 9 edges
  - $E = \{\{A, B\}, \{A, D\}, \{A, E\}, \{B, C\}, \{B, D\}, \{B, E\}, \{C, E\}, \{C, F\}, \{D, E\}\}$

- The **maximum number of edges** in an undirected graph is
  - $E \leq \binom{|V|}{2} = \frac{V(V-1)}{2} = O(V^2)$

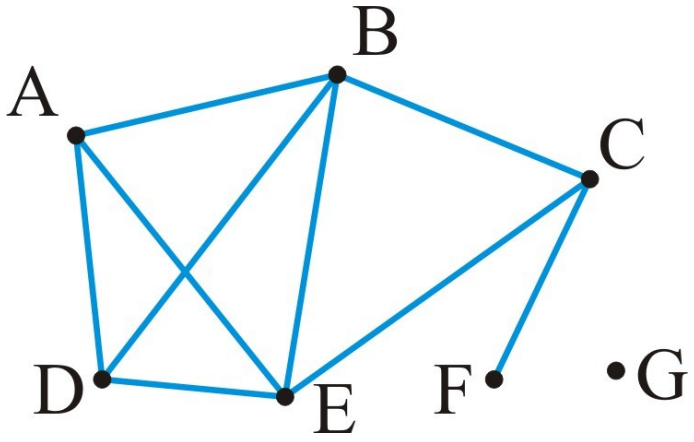
Degree

- **Degree** of a vertex = number of adjacent vertices
  - $degree(A) = degree(D) = degree(C) =$???
  - $degree(B) = degree(E) =$???
  - $degree(F) =$???
  - $degree(G) =$???



Degree

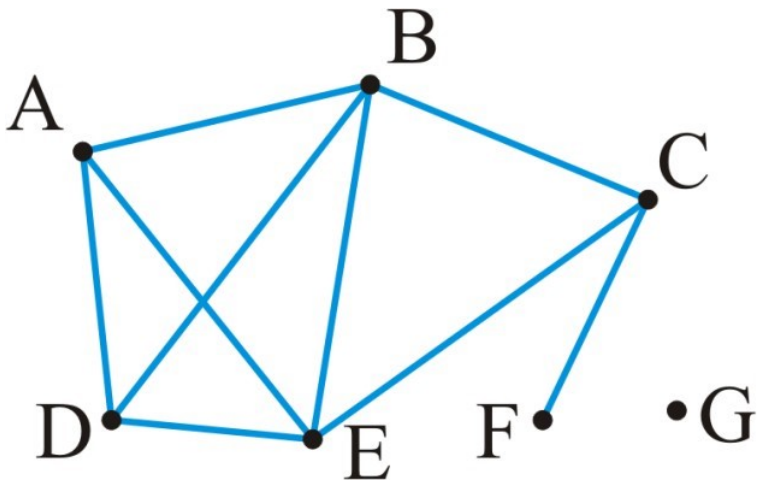
- **Degree** of a vertex = number of adjacent vertices
  - $degree(A) = degree(D) = degree(C) = 3$
  - $degree(B) = degree(E) = 4$
  - $degree(F) = 1$
  - $degree(G) = 0$

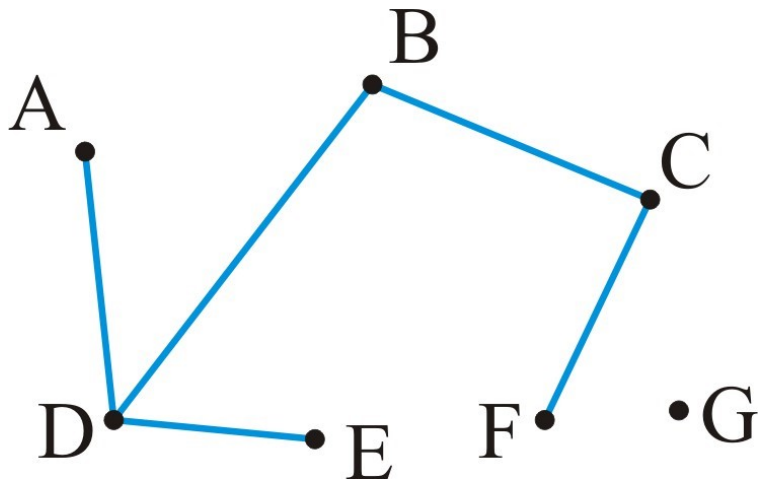


- The vertices adjacent to a given vertex are its ***neighbors***.

Sub-graphs

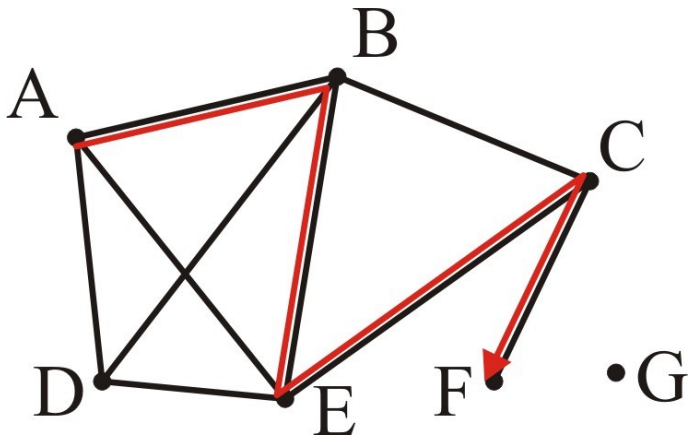
- Sub-graph of a graph = a subset of the vertices and a subset of the edges





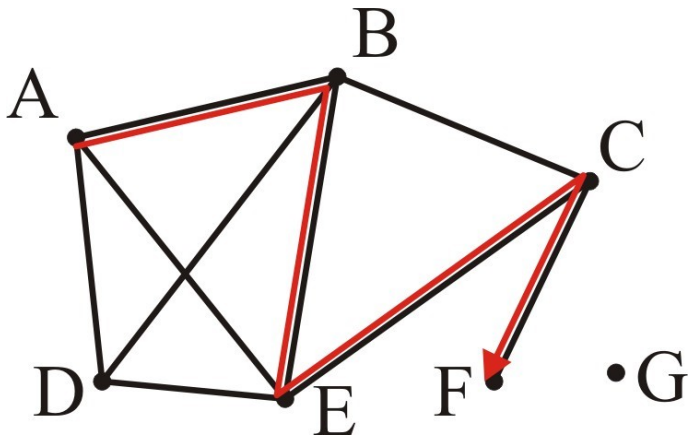
Paths

- Path in an undirected graph: an ordered sequence of vertices $(v_0, v_1, \ldots, v_k)$
  - where $\{v_{j-1}, v_j\}$ is an edge for $j = 1, \ldots, k$.
  - Termed a path from v_j to v_k
- The length of this path is k



Paths

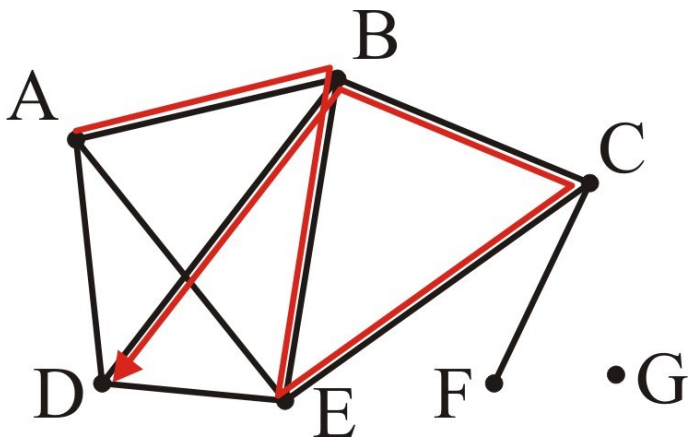
- Path in an undirected graph: an ordered sequence of vertices $(v_0, v_1, \ldots, v_k)$
  - where $\{v_{j-1}, v_j\}$ is an edge for $j = 1, \ldots, k$.
  - Termed a path from v_j to v_k
- The length of this path is k



- **Simple path**: *no repetitions* other than perhaps the first and last vertices
- A simple path of length 4: (A, B, E, C, F)

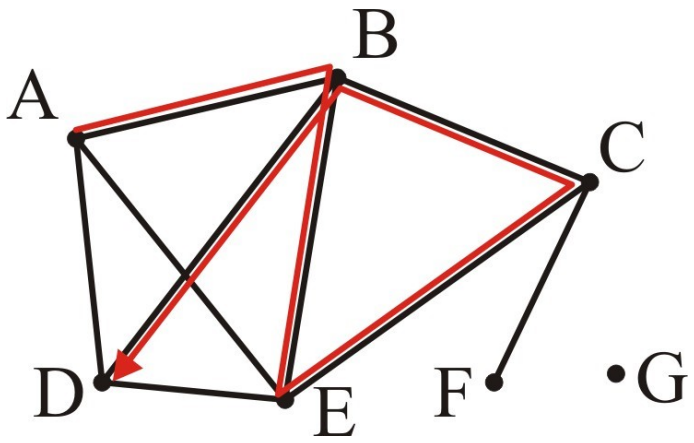
Paths (cont.)

- A path of length 5: (A, B, E, C, B, D)
  - **Is this path simple?**



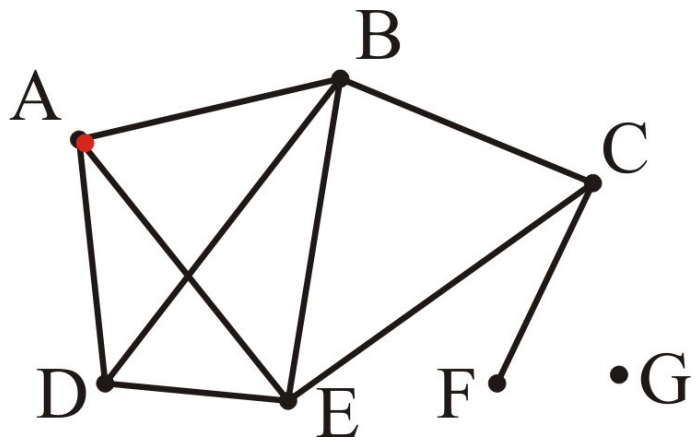
Paths (cont.)

- A path of length 5: (A, B, E, C, B, D)
  - **Is this path simple?**
    - **No !**



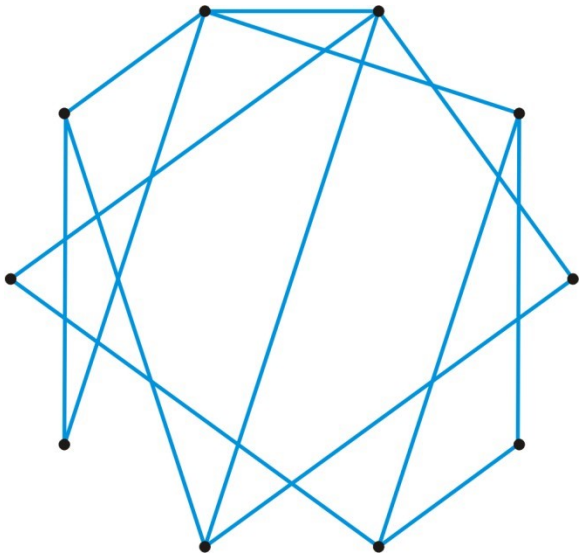
Paths (cont.)

- A trivial path of length 0: (A)

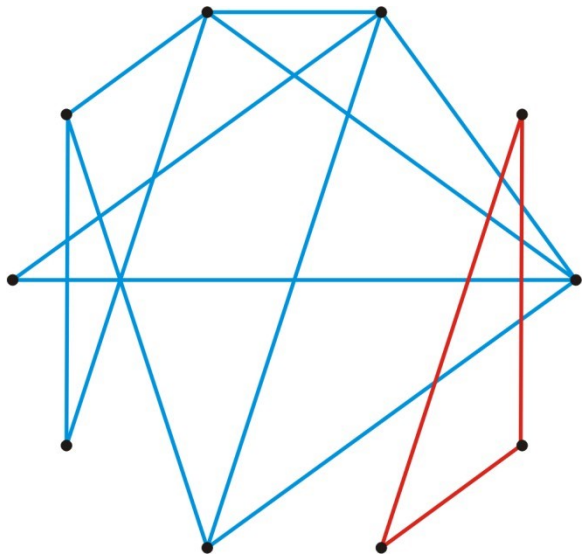


Connectedness

- Two vertices v_i, v_j are **connected** if there exists a path from v_i to v_j
- A **graph is connected** if there exists a path between **any two** vertices



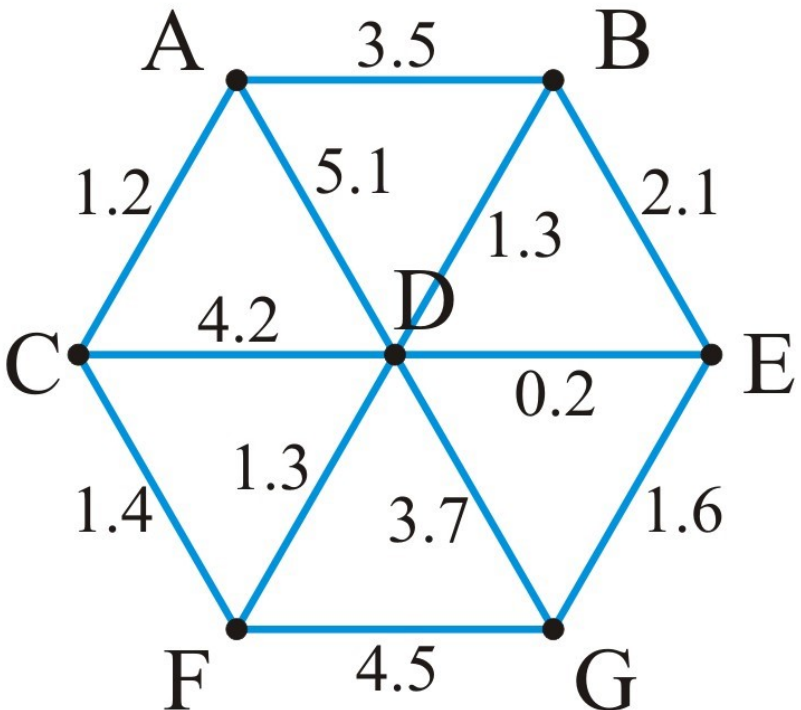
A connected graph



An unconnected graph

Weighted graphs

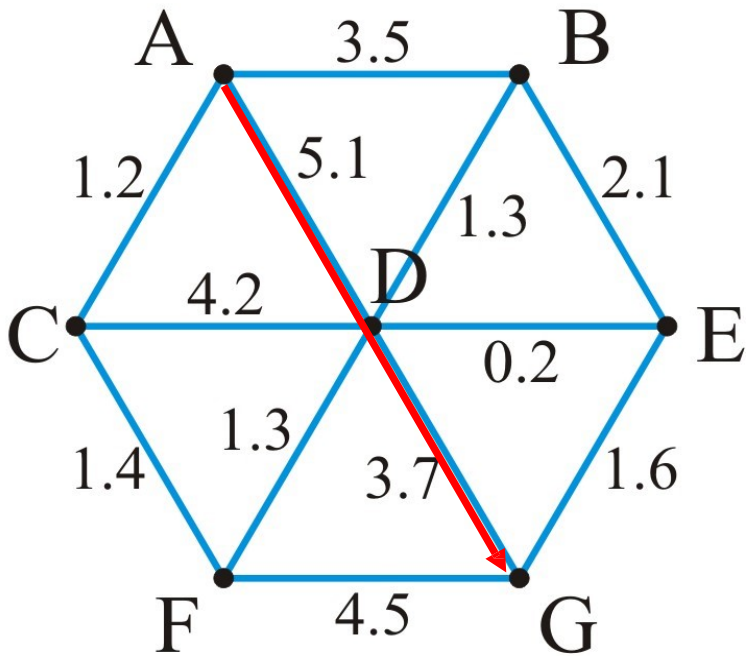
- A **weight** may be associated with each edge in a graph
  - Can represent time, distance, energy consumption, cost, etc.
- Such a graph is called a **weighted graph**



Weighted graphs (cont.)

- **Length of a path** in a weighted graph = sum of all of the edges which make up the path

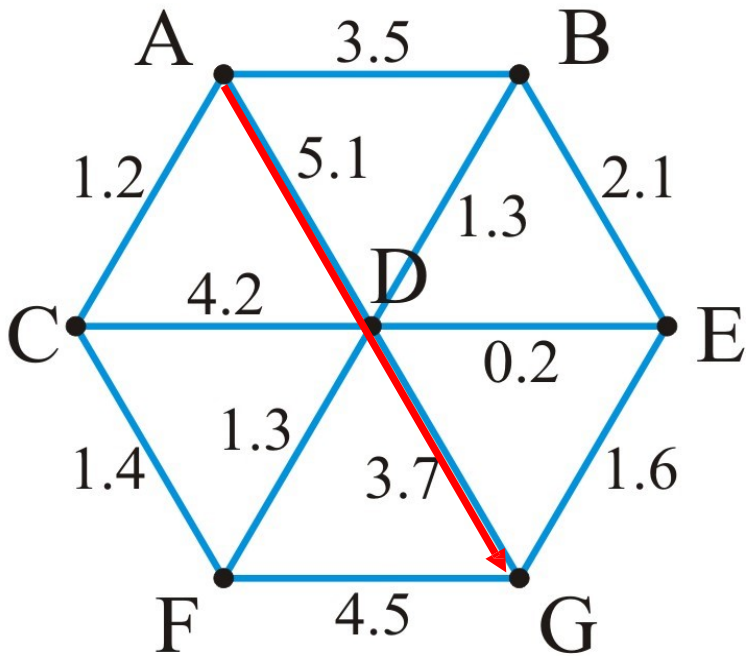
- The length of the path (A, D, G) is ???



Weighted graphs (cont.)

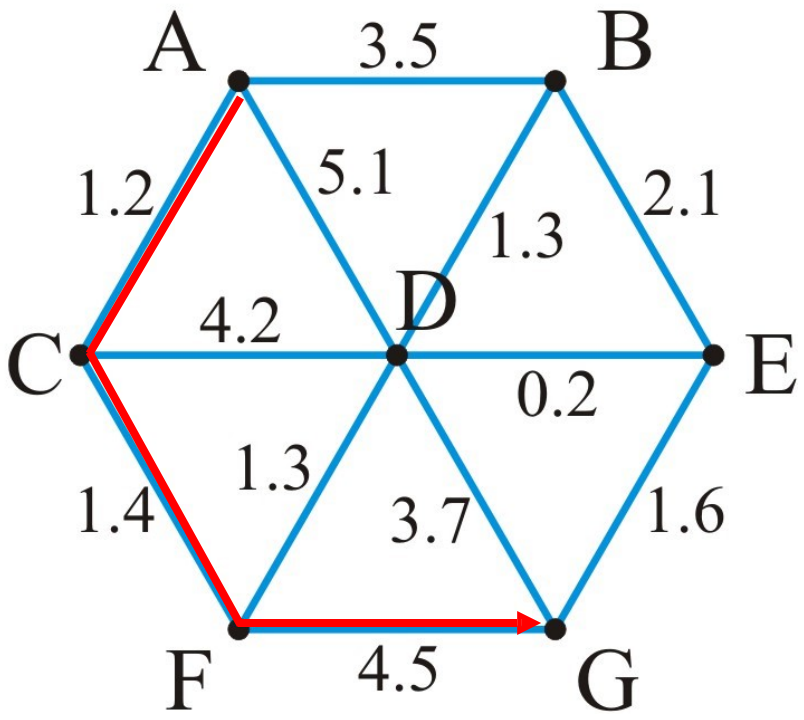
- **Length of a path** in a weighted graph = sum of all of the edges which make up the path

- The length of the path (A, D, G) is $5.1 + 3.7 = 8.8$



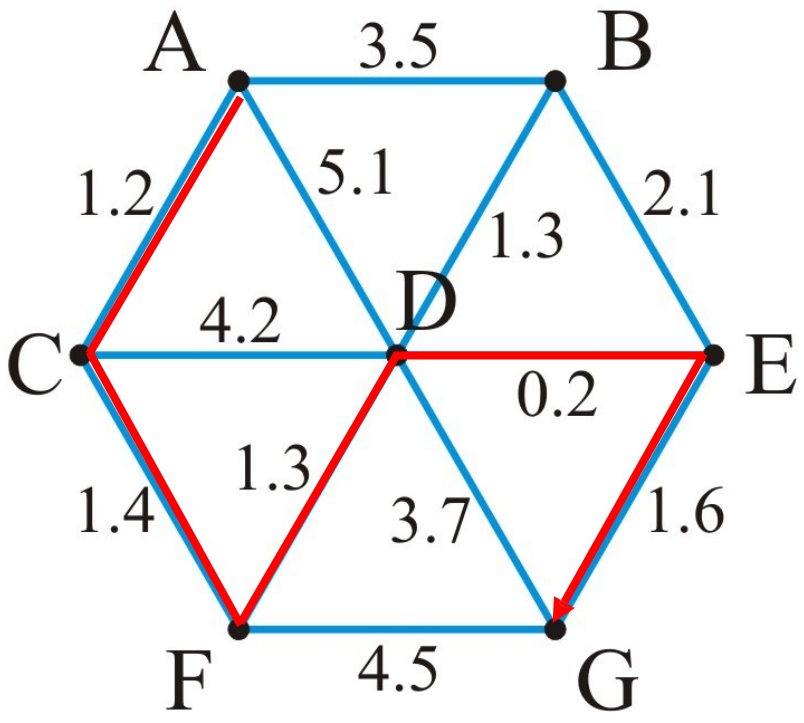
Weighted graphs (cont.)

- Different paths may have different weights
- Another path is (A, C, F, G) with length $1.2 + 1.4 + 4.5 = 7.1$



Weighted graphs (cont.)

- Problem: find the **shortest path** between two vertices
- Here, the shortest path from A to G is (A, C, F, D, E, G) with length 5.7



Directed graphs

- In a directed graph, the edges are associated with a **direction**
- Edges are **ordered pairs** (v_j, v_k) = connection from v_j to v_k
  - Edge (v_j, v_k) is different from the edge (v_k, v_j)

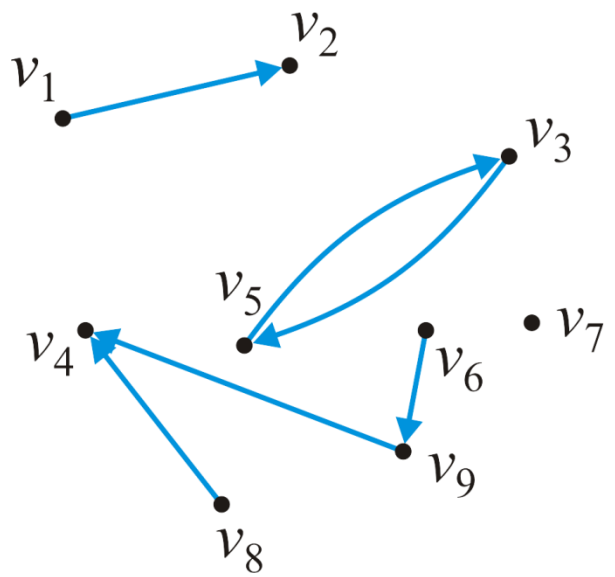
Directed graphs

- In a directed graph, the edges are associated with a **direction**
- Edges are **ordered pairs** (v_j, v_k) = connection from v_j to v_k
  - Edge (v_j, v_k) is different from the edge (v_k, v_j)

- The maximum number of directed edges in a directed graph is:
  - $E \leq 2\binom{|V|}{2} = 2 \times \frac{|V|(|V|-1)}{2} = |V|(|V|-1) = O(|V|^2)$

Directed graphs (cont.)

- Given graph of nine vertices $V = \{v_1, v_2, \ldots v_9\}$
- These six pairs (v_j, v_k) are directed edges
- $E = \{(v_1, v_2), (v_3, v_5), (v_5, v_3), (v_6, v_9), (v_8, v_4), (v_9, v_4)\}$

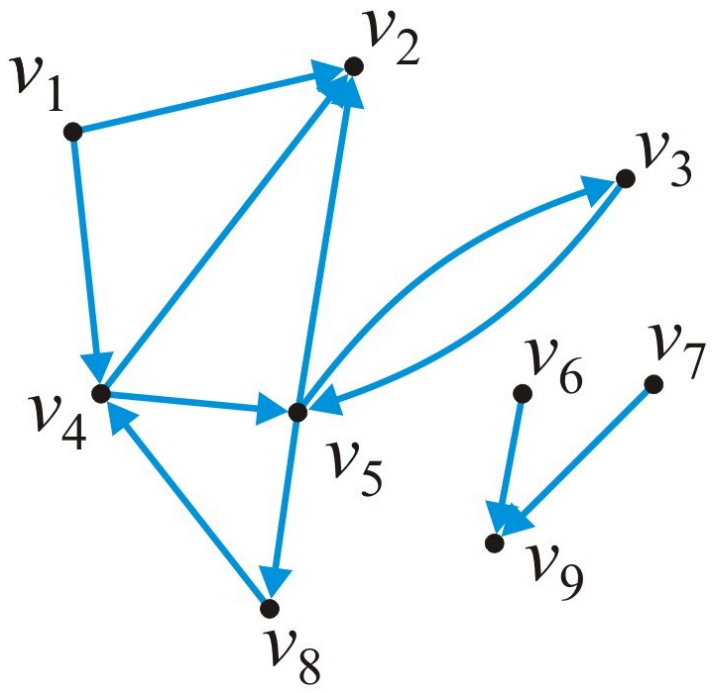


Out-degree & In-degree

- **Out-degree** of a vertex = number of vertices adjacent to the given vertex
- **In-degree** = number of vertices this vertex is adjacent to

- $inDegree\ (v_1) = ???$
- $outDegree\ (v_1) = ???$

- $inDegree\ (v_5) = ???$
- $outDegree\ (v_5) = ???$

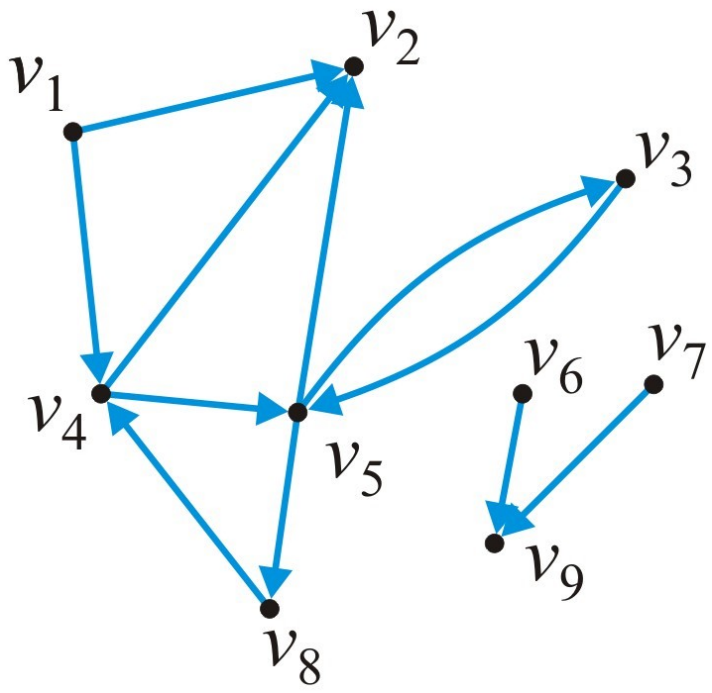


Out-degree & In-degree

- **Out-degree** of a vertex = number of vertices adjacent to the given vertex
- **In-degree** = number of vertices this vertex is adjacent to

- $inDegree\ (v_1)\ =\ 0$
- $outDegree\ (v_1)\ =\ 2$

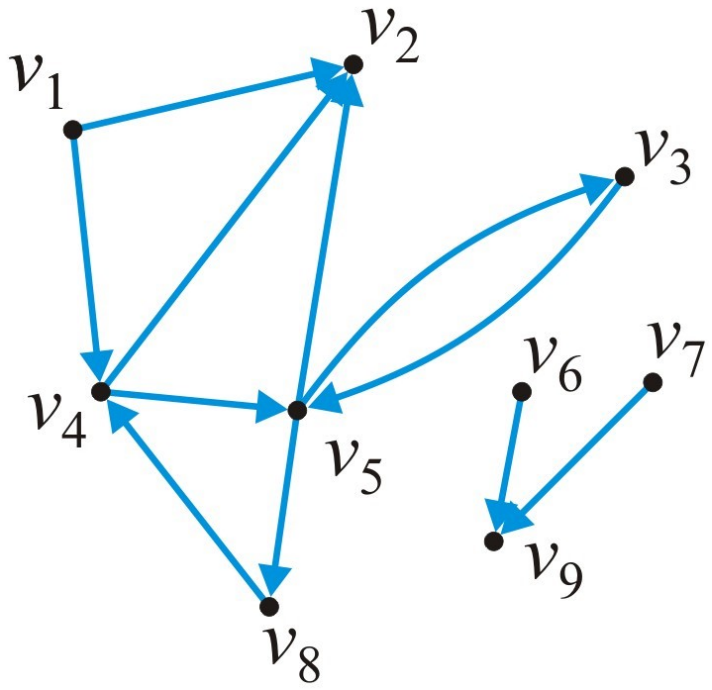
- $inDegree\ (v_5)\ =\ 2$
- $outDegree\ (v_5)\ =\ 3$



Sources and sinks

- **Sources**: Vertices with an in-degree of zero
- **Sinks**: Vertices with an out-degree of zero

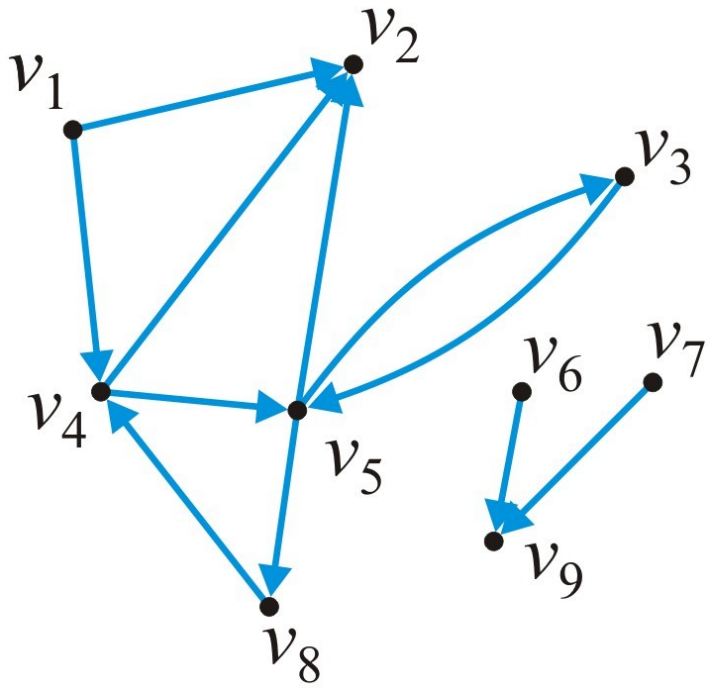
- Sources: ? ? ?
- Sinks: ? ? ?



Sources and sinks

- **Sources**: Vertices with an in-degree of zero
- **Sinks**: Vertices with an out-degree of zero

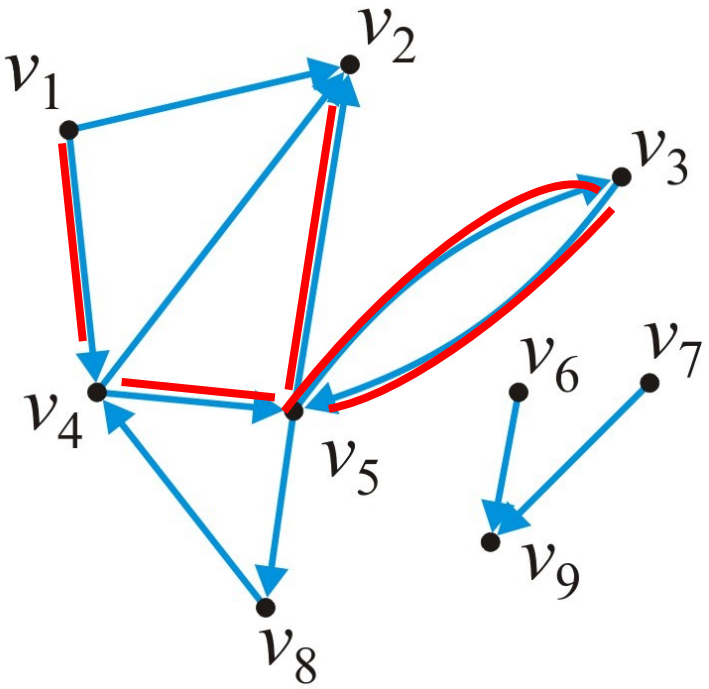
- Sources: v_1, v_6, v_7
- Sinks: v_2, v_9



Paths

- Path in a directed graph: ordered sequence of vertices $(v0, v1, v2, \ldots, vk)$
  - where (v_{j-1}, v_j) is an edge for $j = 1, \ldots, k$

- Path of length 5: $(v1, v4, v5, v3, v5, v2)$

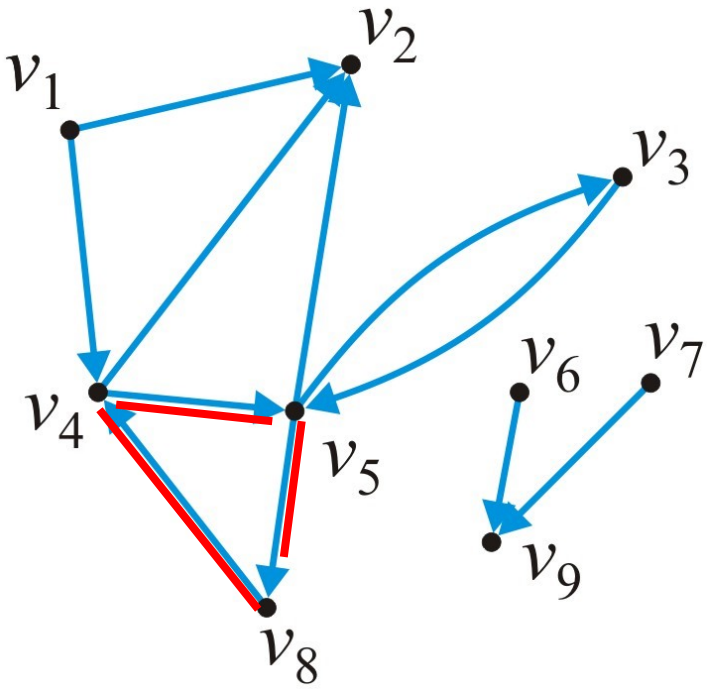


Paths

- Path in a directed graph: ordered sequence of vertices $(v0, v1, v2, \ldots, vk)$
  - where (v_{j-1}, v_j) is an edge for $j = 1, \ldots, k$

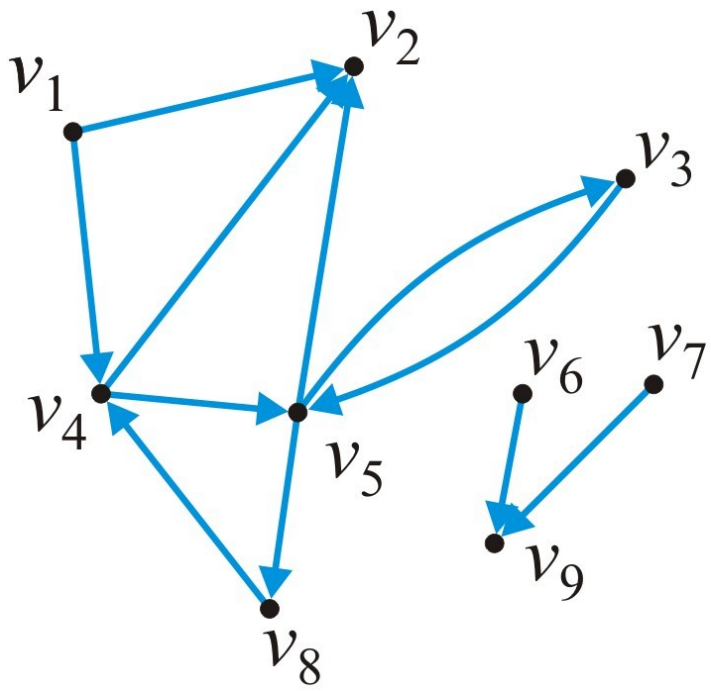
- Path of length 5: $(v1, v4, v5, v3, v5, v2)$

- A simple **cycle** of length 3: $(v8, v4, v5, v8)$



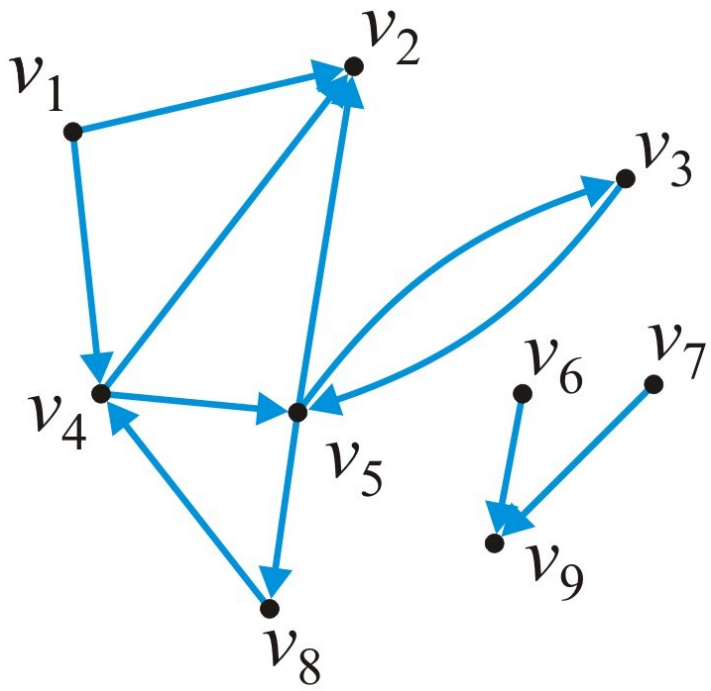
Connectedness

- Two vertices v_j, v_k are connected if there exists a path from v_j to v_k



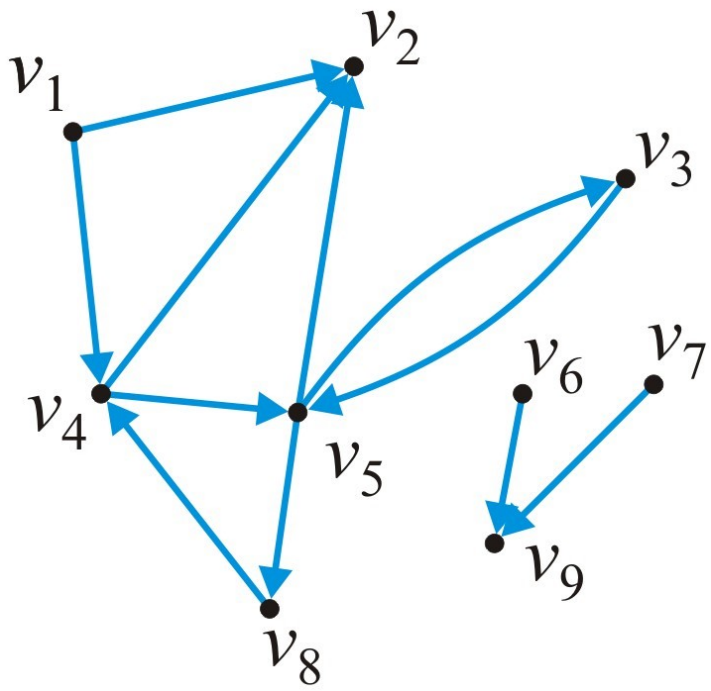
Connectedness

- Two vertices v_j, v_k are connected if there exists a path from v_j to v_k
- A graph is **strongly connected** if there exists a **directed path** between any two vertices



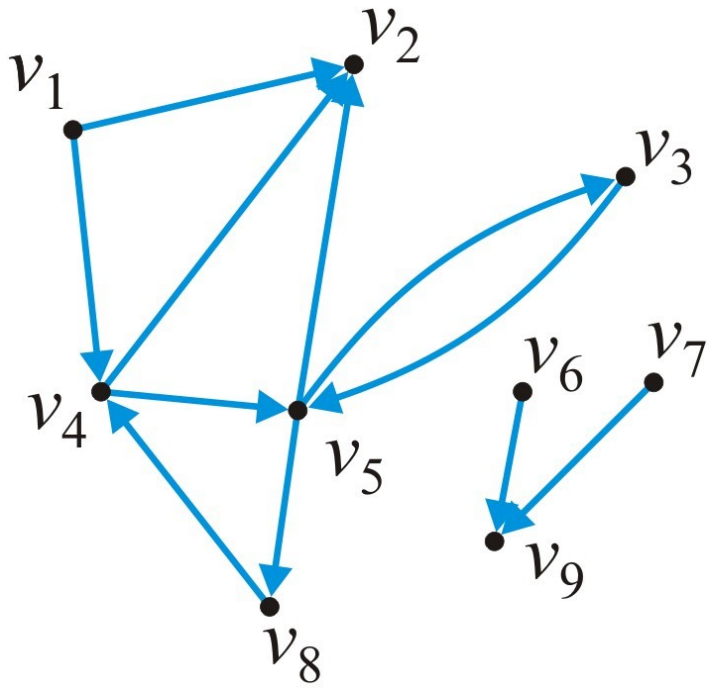
Connectedness

- Two vertices v_j, v_k are connected if there exists a path from v_j to v_k
- A graph is **strongly connected** if there exists a **directed path** between any two vertices
- A graph is **weakly connected** there exists a path between any two vertices that ignores the direction.



Connectedness

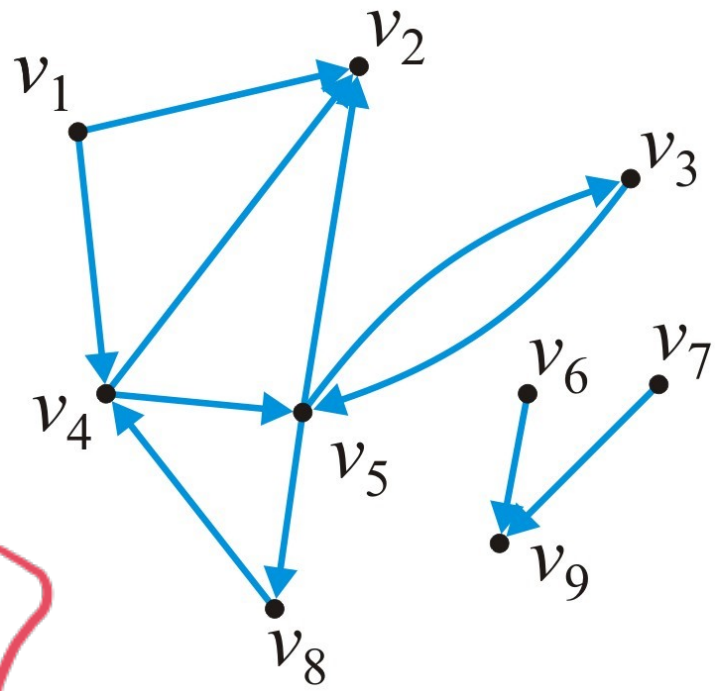
- Two vertices v_j, v_k are connected if there exists a path from v_j to v_k
- A graph is **strongly connected** if there exists a **directed path** between any two vertices
- A graph is **weakly connected** there exists a path between any two vertices that ignores the direction.



- Sub-graph $\{v3, v4, v5, v8\}$ is ??? (strongly/weakly) connected
- Sub-graph $\{v1, v2, v3, v4, v5, v8\}$ is ??? (strongly/weakly) connected.

Connectedness

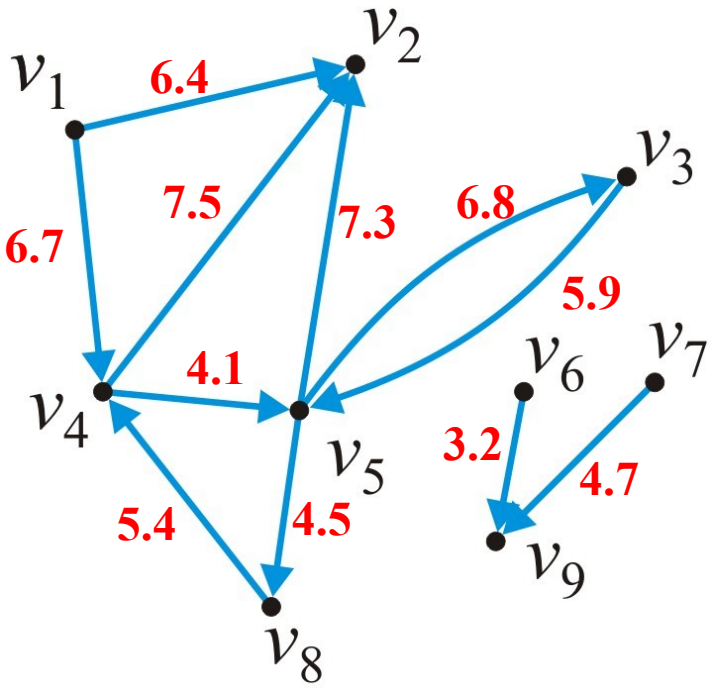
- Two vertices v_j, v_k are connected if there exists a path from v_j to v_k
- A graph is **strongly connected** if there exists a **directed path** between any two vertices
- A graph is **weakly connected** there exists a path between any two vertices that ignores the direction.



- Sub-graph $\{v3, v4, v5, v8\}$ is **strongly** connected
- Sub-graph $\{v1, v2, v3, v4, v5, v8\}$ is **weakly** connected.

Weighted directed graphs

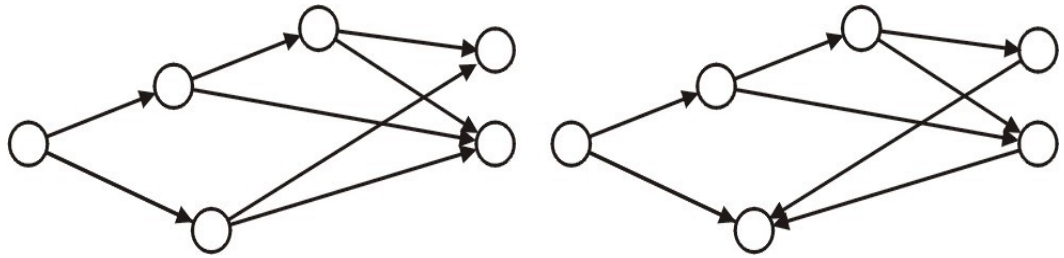
- In a weighted directed graph, each edge is associated with a value
- Unlike weighted undirected graphs:
  - If both (v_j, v_k) and (v_j, v_k) are edges, it is **not required that they have the same weight**



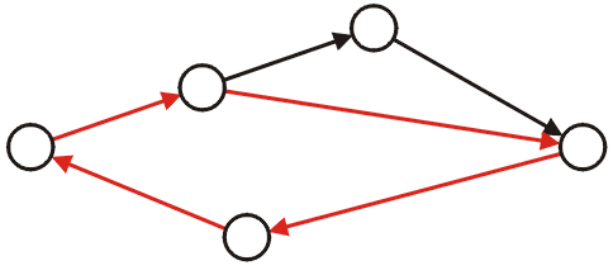
Directed acyclic graphs

- **Directed acyclic graph** is a directed graph which has no cycles (**DAG**s)

- These two are DAGs:

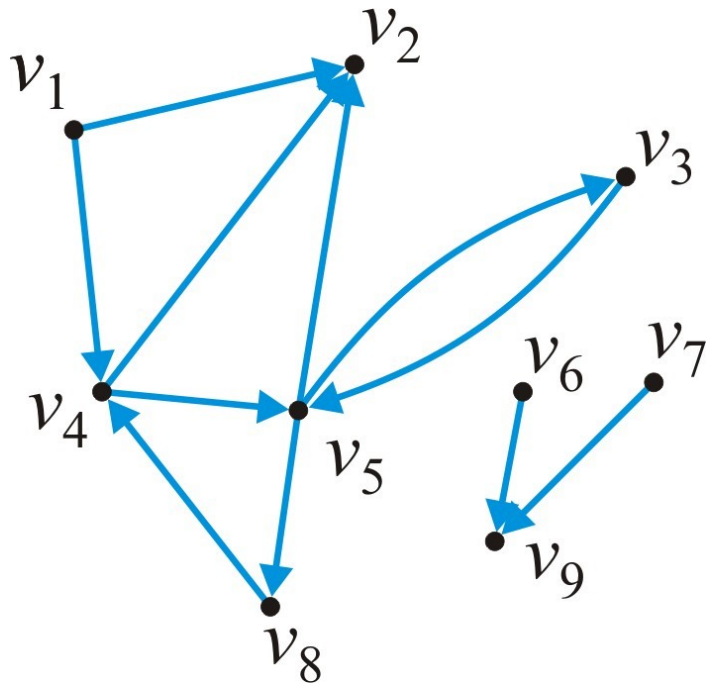


- This directed graph is not acyclic:



Representations

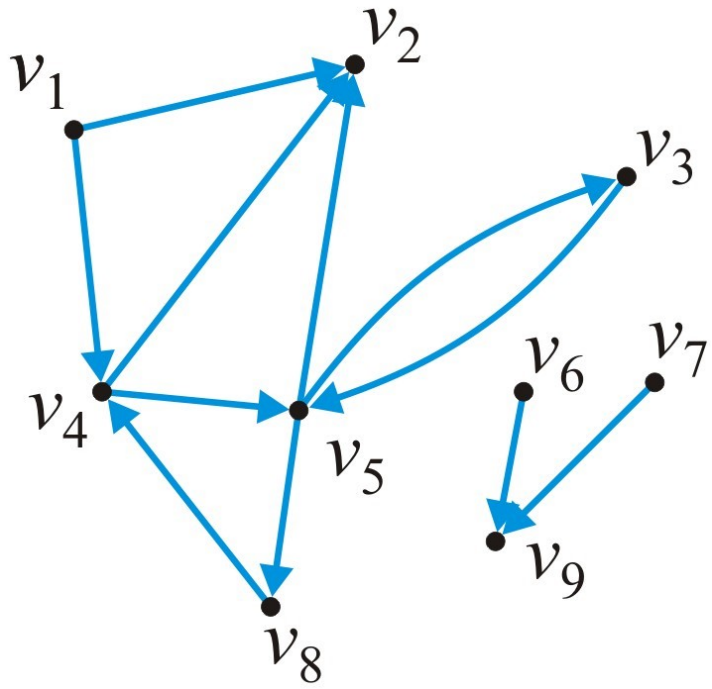
- How do we store the adjacency relations?
  - Binary-relation list
  - Adjacency matrix
  - Adjacency list



Binary-relation list

- The **most inefficient** is a relation list:
  - A container storing the edges
  - $\{(1,2),(1,4),(3,5),(4,2),(4,5),(5,2),(5,3),(5,8),(6,9),(7,9),(8,4)\}$

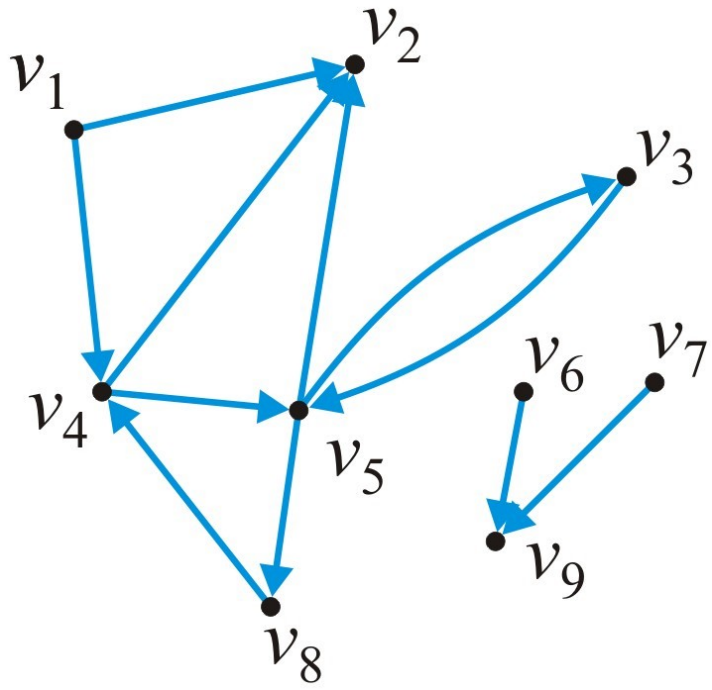
- Requires $O(???)$ memory
- Determine if v_j is adjacent to v_k: $O(???)$
- Find all neighbors of v_j: $O(???)$



Binary-relation list

- The **most inefficient** is a relation list:
  - A container storing the edges
  - $\{(1,2),(1,4),(3,5),(4,2),(4,5),(5,2),(5,3),(5,8),(6,9),(7,9),(8,4)\}$

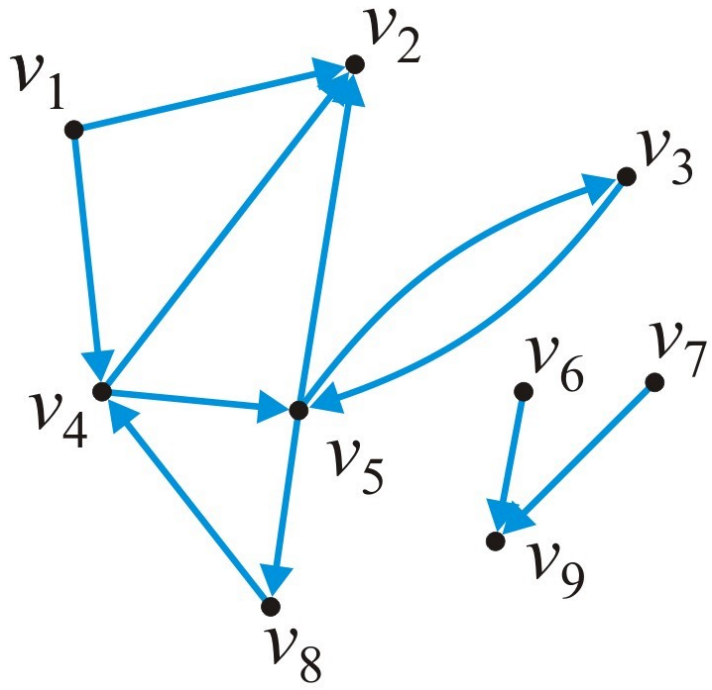
- Requires $O(|E|)$ memory
- Determine if v_j is adjacent to v_k: $O(???)$
- Find all neighbors of v_j: $O(???)$



Binary-relation list

- The **most inefficient** is a relation list:
  - A container storing the edges
  - $\{(1,2),(1,4),(3,5),(4,2),(4,5),(5,2),(5,3),(5,8),(6,9),(7,9),(8,4)\}$

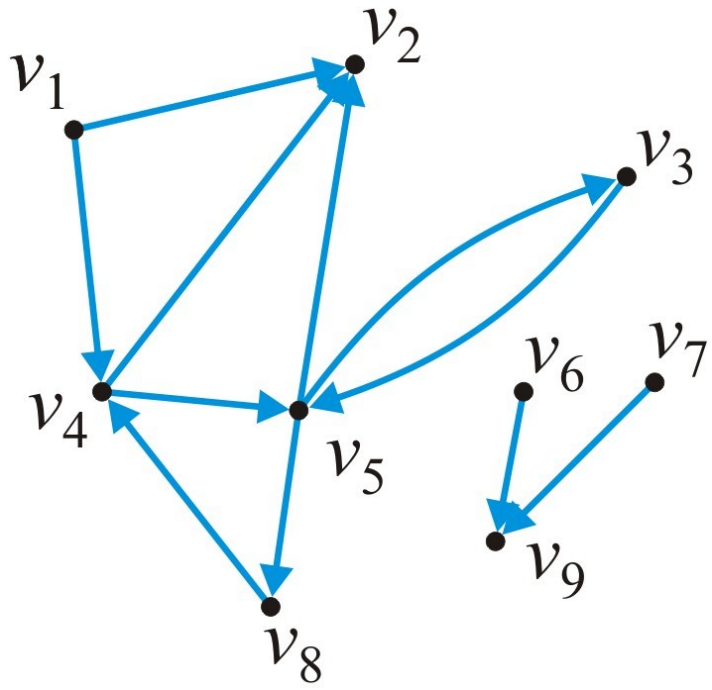
- Requires $O(|E|)$ memory
- Determine if v_j is adjacent to v_k: $O(|E|)$
- Find all neighbors of v_j: $O(???)$



Binary-relation list

- The **most inefficient** is a relation list:
  - A container storing the edges
  - $\{(1,2),(1,4),(3,5),(4,2),(4,5),(5,2),(5,3),(5,8),(6,9),(7,9),(8,4)\}$

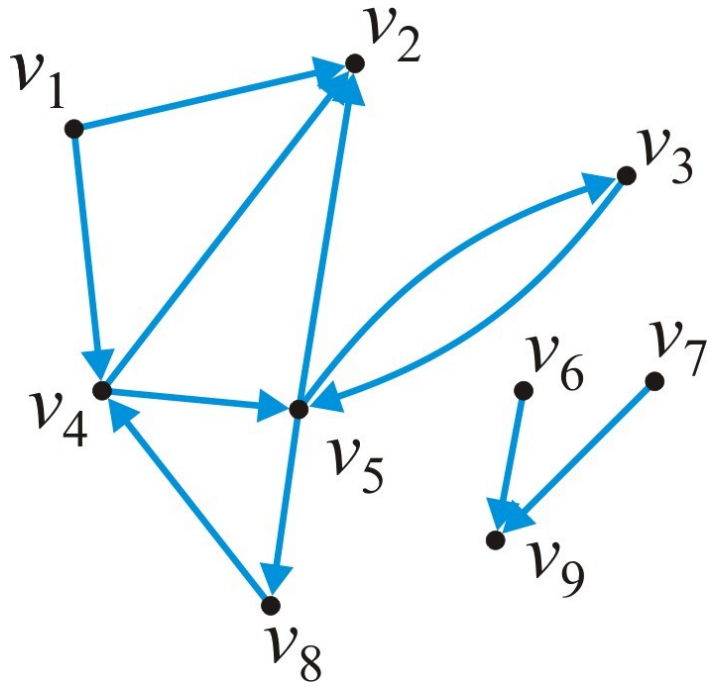
- Requires $O(|E|)$ memory
- Determine if v_j is adjacent to v_k: $O(|E|)$
- Find all neighbors of v_j: $O(|E|)$



Adjacency matrix

- Requiring **more memory** but also **faster**, an adjacency matrix
- The matrix entry (j, k) is set to true if there is an edge (v_j, v_k)

- Requires $O(???)$ memory
- Determine if v_j is adjacent to v_k: $O(???)$
- Find all neighbors of v_j: $O(???)$

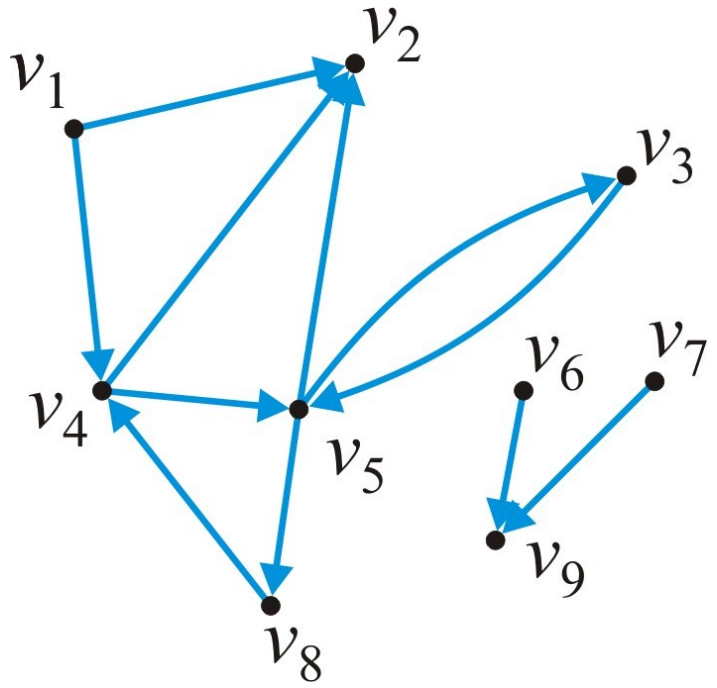


	1	2	3	4	5	6	7	8	9
1		T		T					
2									
3					T				
4		T			T				
5		T	T					T	
6									T
7									T
8				T					
9									

Adjacency matrix

- Requiring **more memory** but also **faster**, an adjacency matrix
- The matrix entry (j, k) is set to true if there is an edge (v_j, v_k)

- Requires $O(|V|^2)$ memory
- Determine if v_j is adjacent to v_k: $O(???)$
- Find all neighbors of v_j: $O(???)$

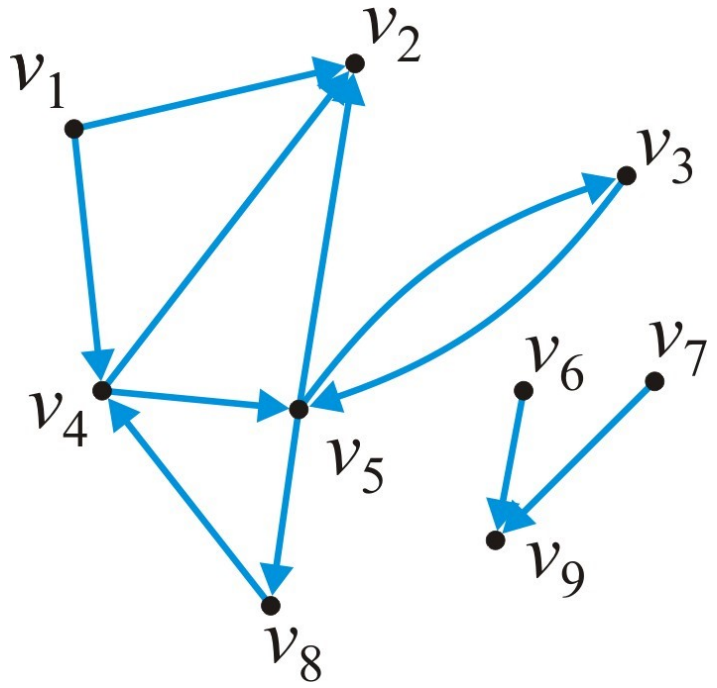


	1	2	3	4	5	6	7	8	9
1		T		T					
2									
3					T				
4		T			T				
5		T	T					T	
6									T
7									T
8				T					
9									

Adjacency matrix

- Requiring **more memory** but also **faster**, an adjacency matrix
- The matrix entry (j, k) is set to true if there is an edge (v_j, v_k)

- Requires $O(|V|^2)$ memory
- Determine if v_j is adjacent to v_k: $O(1)$
- Find all neighbors of v_j: $O(???)$

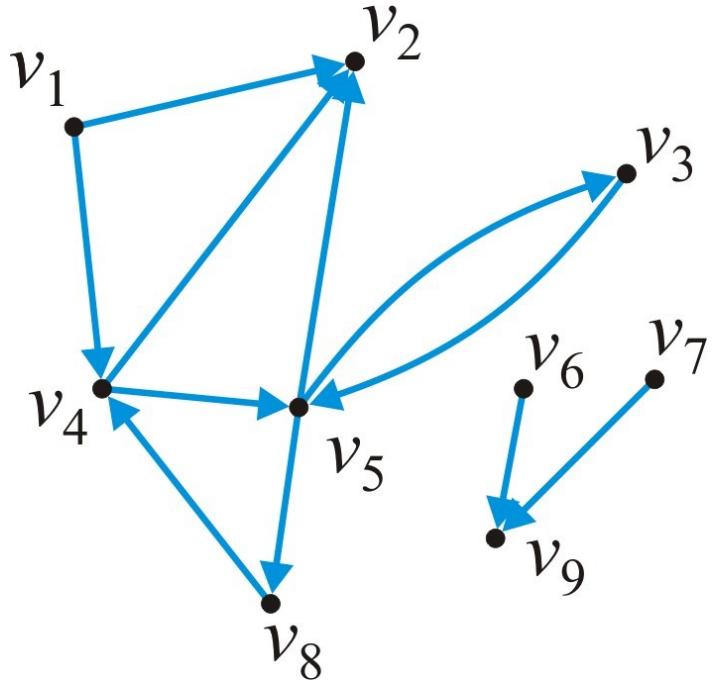


	1	2	3	4	5	6	7	8	9
1		T		T					
2									
3					T				
4		T			T				
5		T	T					T	
6									T
7									T
8				T					
9									

Adjacency matrix

- Requiring **more memory** but also **faster**, an adjacency matrix
- The matrix entry (j, k) is set to true if there is an edge (v_j, v_k)

- Requires $O(|V|^2)$ memory
- Determine if v_j is adjacent to v_k: $O(1)$
- Find all neighbors of v_j: $O(|V|)$

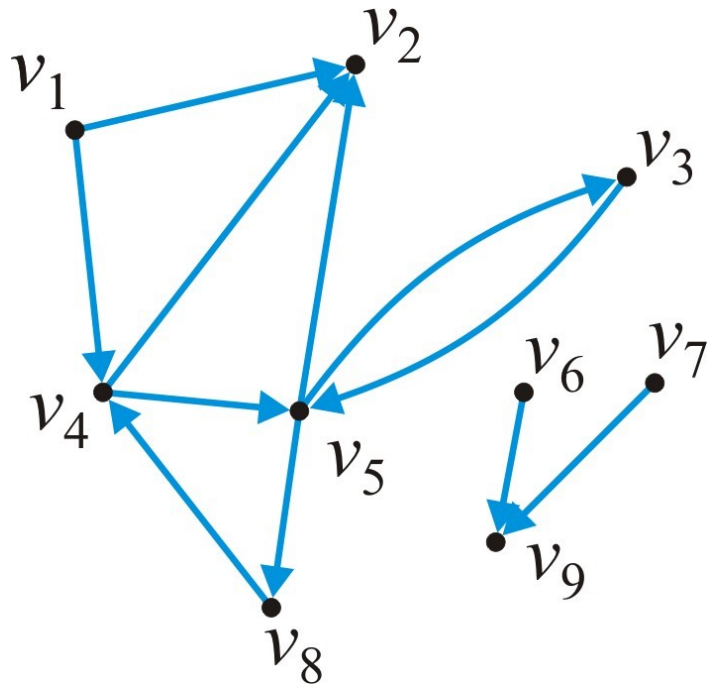


	1	2	3	4	5	6	7	8	9
1		T		T					
2									
3					T				
4		T			T				
5		T	T					T	
6									T
7									T
8				T					
9									

Adjacency list

- **Most efficient** for algorithms is an adjacency list
- Each vertex is associated with a list of its neighbors

1 • → 2 → 4
2 •
3 • → 5
4 • → 2 → 5
5 • → 2 → 3 → 8
6 • → 9
7 • → 9
8 • → 4
9 •



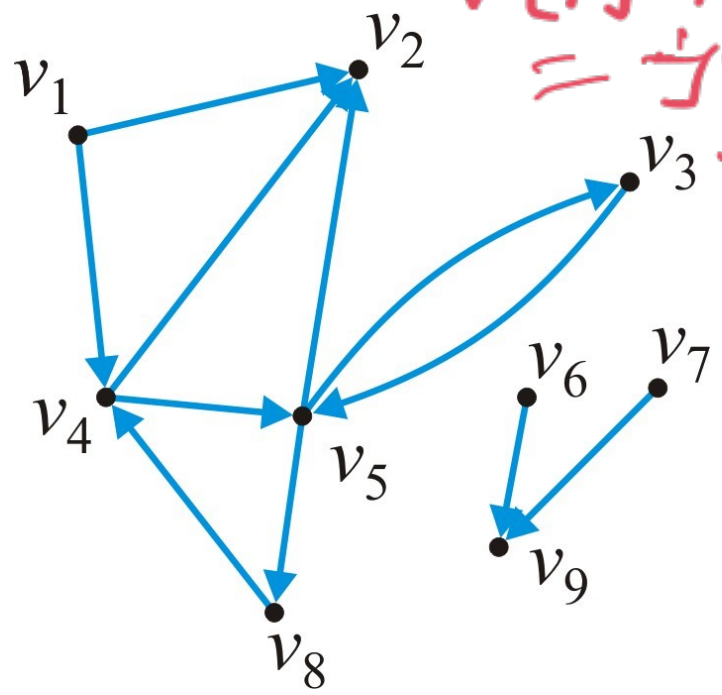
- Requires $O(???)$ memory
- On average:
  - Determining if v_j is adjacent to v_k is $O(???)$
  - Find all neighbors of v_j is $O(???)$

v.first = v.second

Adjacency list

- **Most efficient** for algorithms is an adjacency list
- Each vertex is associated with a list of its neighbors

1 • → 2 → 4
2 •
3 • → 5
4 • → 2 → 5
5 • → 2 → 3 → 8
6 • → 9
7 • → 9
8 • → 4
9 •

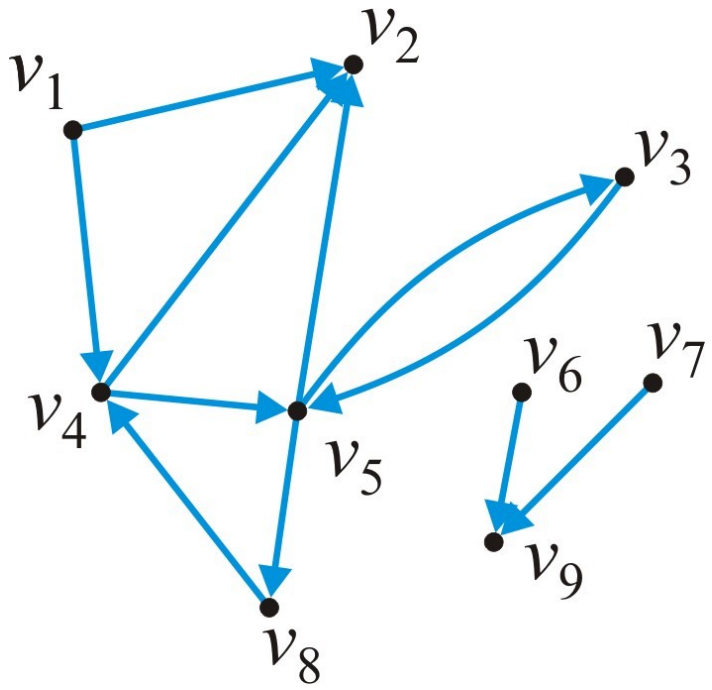


- Requires $O(|V| + |E|)$ memory
- On average:
  - Determining if v_j is adjacent to v_k is $O(???)$
  - Find all neighbors of v_j is $O(???)$

Adjacency list

- **Most efficient** for algorithms is an adjacency list
- Each vertex is associated with a list of its neighbors

1 • → 2 → 4
2 •
3 • → 5
4 • → 2 → 5
5 • → 2 → 3 → 8
6 • → 9
7 • → 9
8 • → 4
9 •

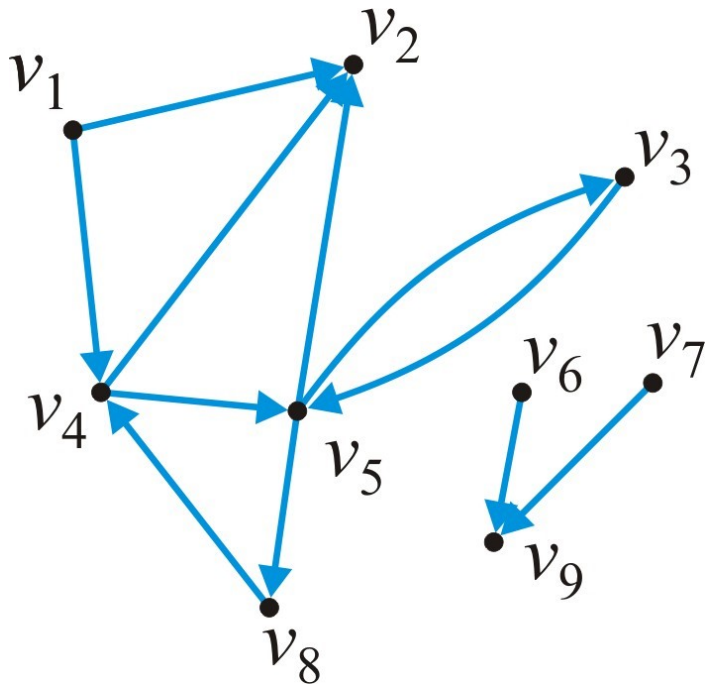


- Requires $O(|V| + |E|)$ memory
- On average:
  - Determining if v_j is adjacent to v_k is $O(|E|/|V|)$
  - Find all neighbors of v_j is $O(???)$

Adjacency list

- **Most efficient** for algorithms is an adjacency list
- Each vertex is associated with a list of its neighbors

1 • → 2 → 4
2 •
3 • → 5
4 • → 2 → 5
5 • → 2 → 3 → 8
6 • → 9
7 • → 9
8 • → 4
9 •



- Requires $O(|V| + |E|)$ memory
- On average:
  - Determining if v_j is adjacent to v_k is $O(|E|/|V|)$
  - Find all neighbors of v_j is $O(|E|/|V|)$

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports **operation 1** more efficiently	

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports **operation 1** more efficiently	Supports **operation 2** more efficiently

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports **operation 1** more efficiently	Supports **operation 2** more efficiently
Space requirement: $O(???)$	Space requirement: $O(???)$

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports operation 1 more efficiently	Supports operation 2 more efficiently
Space requirement: $O(\|V\|^2)$	Space requirement: $O(\|E\| + \|V\|)$, **linear** in the size of the graph

Adjacency Matrix vs Adjacency List

- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports operation 1 more efficiently	Supports operation 2 more efficiently
Space requirement: $O(\|V\|^2)$	Space requirement: $O(\|E\| + \|V\|)$, **linear** in the size of the graph
Better if the graph is **dense/sparse**?	Better if the graph is **dense/sparse**?

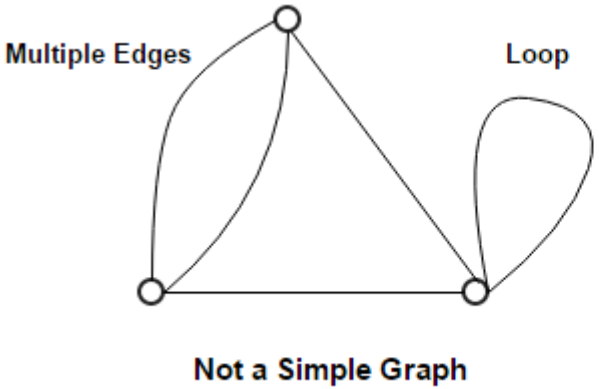
Adjacency Matrix vs Adjacency List

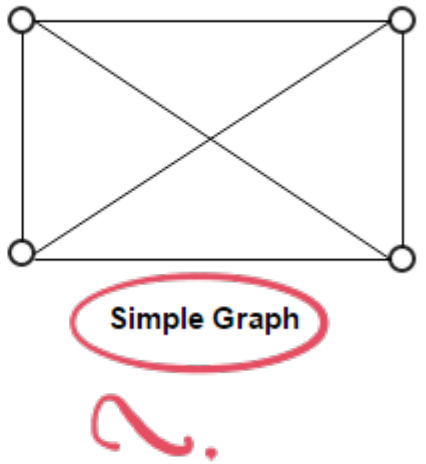
- Two common graph operations:
  1. Determine **whether there is an edge** from v_i to v_j.
  2. Find **all vertices adjacent** to a given vertex v_i.

Adjacency matrix	**Adjacency list**
Supports operation 1 more efficiently	Supports operation 2 more efficiently
Space requirement: $O(\|V\|^2)$	Space requirement: $O(\|E\| + \|V\|)$, **linear** in the size of the graph
Better if the graph is **dense** (too many edges)	Better if the graph is **sparse** (few edges)

Definitions

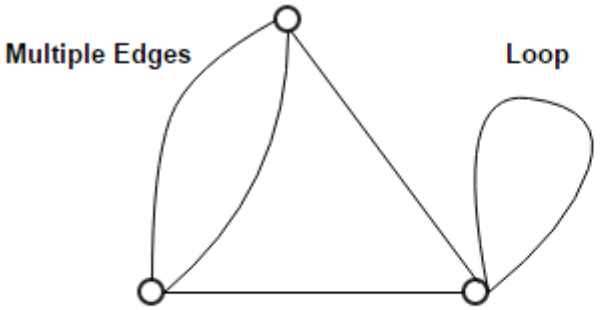
- **Simple Graph**: no self loop + no parallel edge



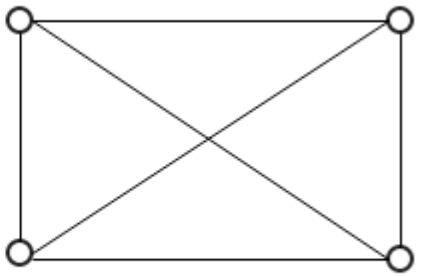


Definitions

- **Simple Graph**: no self loop + no parallel edge



Not a Simple Graph



Simple Graph

- **Null Graph**: a graph without any edge
- **Isolated vertex**: $degree(v_i) = 0$
- **Pendent vertex**: $degree(v_i) = 1$

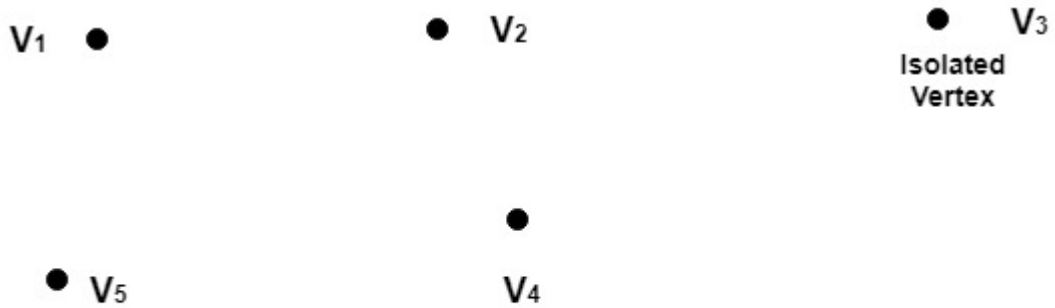
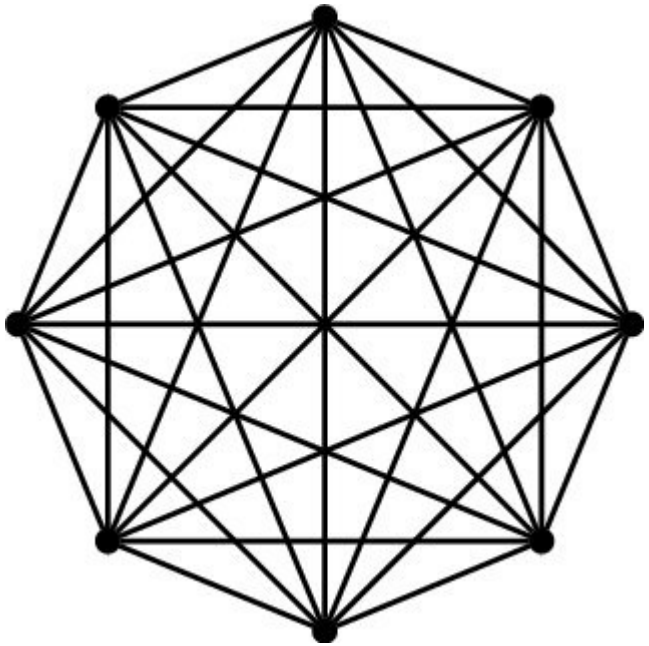


Fig:Null Graph

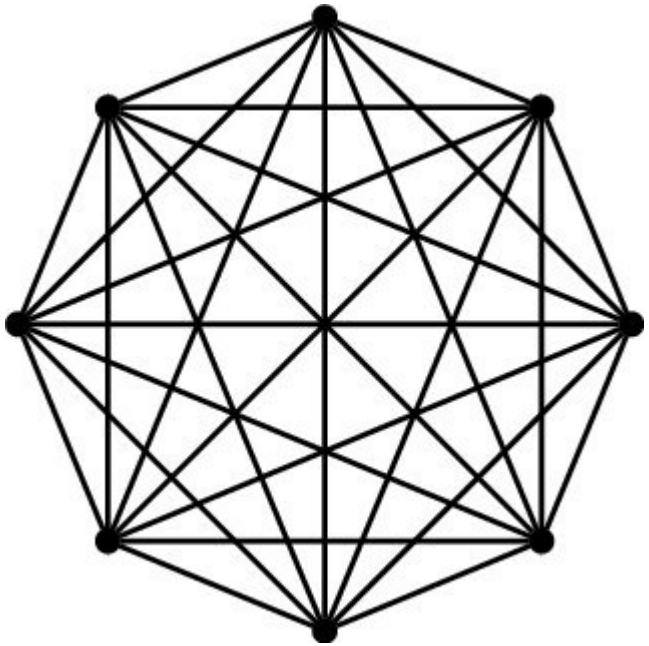
Definitions

- **Complete graph**: each vertex connected with every vertices. Represented by K_n.

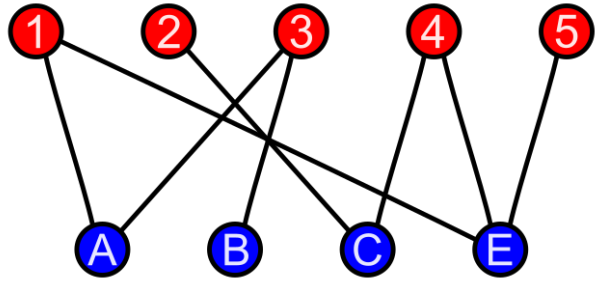


Definitions

- **Complete graph**: each vertex connected with every vertices. Represented by K_n.

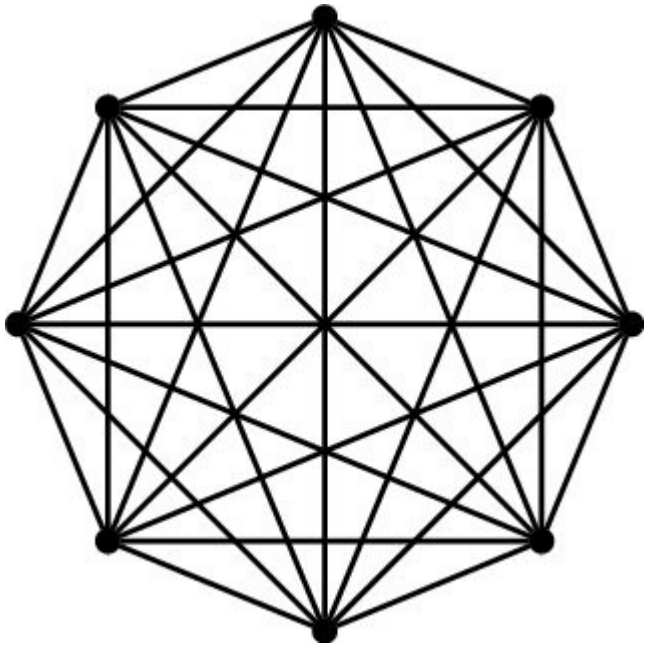


- **Bipartite graph**: Represented by $K_{n,m}$

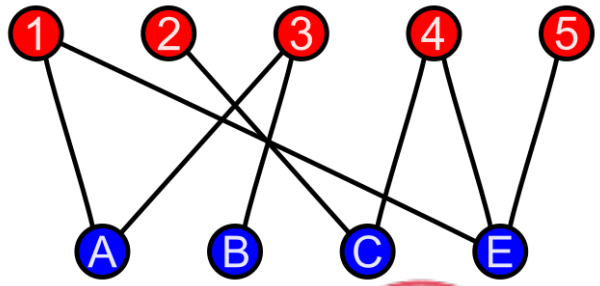


Definitions

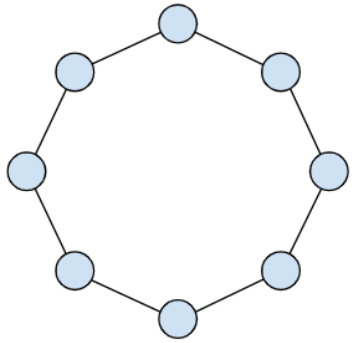
- **Complete graph**: each vertex connected with every vertices. Represented by K_n.



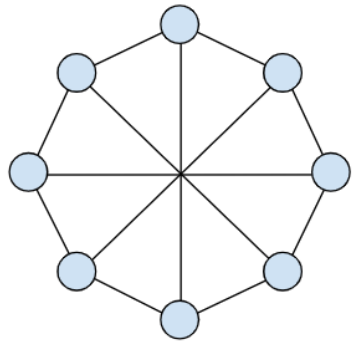
- **Bipartite graph**: Represented by $K_{n,m}$



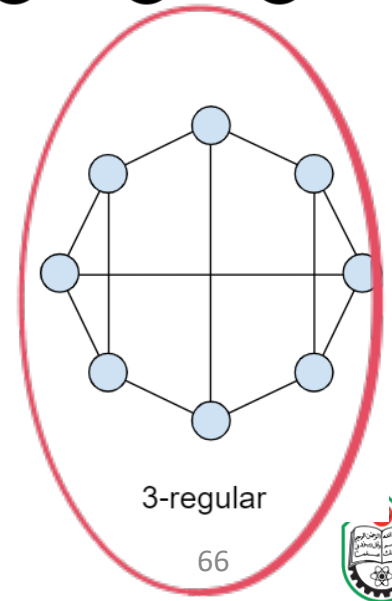
- **Regular graph**: same degree of each vertex.



2-regular



3-regular



3-regular

- Reference text:
  - Chapter 9, Data Structures & Algorithm Analysis in C++, 4th Edition, by Mark Weiss